

One sentence. SCC treats legality as a runtime invariant: a system that cannot prove the legality of its next step has no right to take it.

Layer		Role	Primary evidence
Full SCC Canon		Formal authority: state spaces, hard clauses, proof obligations, audit determinism, lineage, refinement, federation assumptions, mechanization targets, and claim boundaries.	Articles, assumption ledger, mechanization targets.
Engineering book	Hand-	Builder discipline: small deterministic kernel first, checker outside product logic, proof-obligation bundles, audit/replay, TCB hardening, release gates, handoff rules.	Build order, runbooks, TCB ledger, CI/CD gates.
Kernel Fortress		Executable boundary artifact: checks one candidate transition before state acceptance using previous state, next state, event, audit event, proof-obligation bundle, and descriptor.	Rust checker, TypeScript reference, golden vectors, negative corpus, CI transcript.
Formal bridge		Mechanization layer for the implemented v1 subset: Bool-to-Prop gate soundness, seven-stage witness completeness, and quorum-intersection arithmetic.	Lean/Coq files, mechanization manifest, formal-gate script.
Builder Scaffold		Minimal starting implementation shape for teams adopting SCC discipline before handing off to Kernel Fortress-style evidence.	Python runtime skeleton, 16 tests, runbook, scaffold manifest.

Evidence flow.

Canon clause → Handbook rule → Kernel Fortress gate → Fixture → Release transcript → Reviewer claim.

What is now locked down.

- Mechanization is no longer only aspirational: the package includes Lean/Coq bridge files and a formal-gate script for the implemented v1 subset.
- Rust/TypeScript stress coverage now includes memory-compression lineage, numerical simplex bounds, and federation quorum arithmetic.
- Distributed SCC is framed as a certificate layer over the single-node gate: quorum certificates, cross-shard lineage, and bounded halt propagation.
- Numerical stability is named in the TCB ledger: finite binary64 encoding, negative-zero normalization, descriptor epsilon, drift bounds, and threshold margins.

Non-claim. These artifacts do not by themselves prove general AI safety, compiler correctness, operating-system correctness, SHA-256 security, semantic harmlessness, or the correctness of every possible SCC implementation.

Reviewer reading order: one-pager → NeurIPS E&D paper → supplement README → release transcript → formal bridge → Canon/Handbook appendices.